
TRABAJO PRÁCTICO 1

PROBLEMA 1 – FUERZA BRUTA vs. BACKTRACKING

Diseñar e implementar un algoritmo que resuelve el problema de la mochila mediante fuerza bruta.

Luego, modificarlo para que, mediante backtracking llegue al mismo resultado en menos tiempo.

- Calcular el orden de complejidad temporal de los algoritmos desarrollados.
- Aplicarlos a diferentes sets de datos obtenidos con el código adjunto crear_mochila.py, de tamaños adecuados para poder comparar la curva de tiempos de ejecución teórica y real en ambos algoritmos.

PROBLEMA 2 – GREEDY

Diseñar e implementar un algoritmo greedy que resuelve el problema de la mochila. Dicho algoritmo debe tener garantía de calidad 1/2.

- Calcular el orden de complejidad temporal del algoritmo desarrollado.
- Aplicarlo a diferentes sets de datos obtenidos con el código adjunto crear_mochila.py, de tamaños adecuados para poder comparar la curva de tiempos de ejecución teórica y real.
- Aplicarlo a un set de datos creado manualmente donde actúe la garantía de calidad.

PROBLEMA 3 – PROGRAMACIÓN DINÁMICA

Diseñar e implementar dos algoritmos que resuelvan el problema de la mochila mediante programación dinámica. Uno de los algoritmos deberá resolverlo con el planteo tradicional (maximizar el beneficio para un capacidad fija) y el otro deberá resolverlo con el planteo alternativo (Minimizar el peso para un beneficio fijo).

- Calcular el orden de complejidad temporal de los algoritmos desarrollados.
- Aplicarlos a diferentes sets de datos obtenidos con el código adjunto crear_mochila.py, de tamaños adecuados para poder comparar la curva de tiempos de ejecución teórica y real en ambos algoritmos.
- Aplicarlos ambos algoritmos a diferentes sets de datos obtenidos con el código adjunto crear_mochila.py, de tamaños adecuados para poder comparar los tiempos de ejecución de ambos algoritmos. Analizar los resultados obtenidos.

PROBLEMA 4 – PROGRAMACIÓN LINEAL

Diseñar e implementar un algoritmo que resuelvan el problema de la mochila mediante programación lineal utilizando utilizando la biblioteca Pulp.

- Calcular el orden de complejidad temporal del algoritmo desarrollado.

- Aplicarlos a diferentes sets de datos obtenidos con el código adjunto crear_mochila.py, de tamaños adecuados para poder comparar la curva de tiempos de ejecución teórica y real en ambos algoritmos.

CONCLUSIÓN

Comparar en los rangos en que sea posible los tiempos obtenidos para los mismos problemas por los 6 algoritmos desarrollados. Analizar los resultados obtenidos.

Condiciones Generales de los problemas

Para cada algoritmo, se debe incluir:

1. Supuestos: identificar supuestos, condiciones, limitaciones y/o premisas bajo los cuales funcionará el algoritmo desarrollado
2. Diseño:
 - a. Incluir un Pseudocódigo
 - b. Detallar las estructuras de datos utilizadas
2. Seguimiento: Ejemplo de seguimiento con un set de datos reducido
3. Complejidad: Análisis de la complejidad temporal a partir del pseudocódigo
4. Sets de datos: Adjuntar los sets de datos utilizados. Indicar los criterios con que se tomaron.
5. Tiempos de Ejecución: medir los tiempos de ejecución de cada set de datos y presentarlos en un gráfico.
6. Informe de Resultados: redactar un informe de resultados comparando los tiempos de ejecución. ¿Se corresponde con la complejidad determinada inicialmente?. El gráfico comparativo de tiempos debe incluir tanto la curva con los valores medidos como la curva teórica de ajuste.

Condiciones Generales de Entrega

- El trabajo debe ser entregado en un archivo zip conteniendo:
 - Documento con carátula, índice y numeración de páginas. La carátula debe incluir nombre y padrón de los integrantes del grupo. Debe presentarse en formato PDF.
 - Archivos con los fuentes desarrollados. Indicar en el documento el lenguaje de programación utilizado e incluir instrucciones para compilar (de ser necesario) y ejecutar.
 - Archivos con los sets de datos utilizados
 - Archivos con resultados obtenidos para cada set de datos
- Si se incluyeran referencias bibliográficas, utilizar normas APA 7ma edición